

PI Documunetation

Razvan Barna

May 2026

Contents

1	Introduction	2
1.1	Contextul temei și probleme rezolvate	2
1.2	Obiectivele proiectului	2
2	Fundamentare Teoretică	2
2.1	Detectia facială	2
2.2	Recunoasterea fetei:	4
3	Proiectare si implementare	4
3.1	Descrierea solutiei alese si a modului de implementare	5
3.2	Descrierea funcționalităților modulelor implementate	7
3.3	Pipeline-ul Complet al Sistemului	7
3.4	Modul de utilizare al aplicației implementate	8
4	Rezultate Experimentale	9
4.1	Evaluarea modului de detecție facială	9
4.1.1	Analiză calitativă în timp real	9
4.1.2	Pipeline-ul de testare automatizată	9
4.1.3	Rezultate quantitative și comparație (IoU vs. Precizie)	9
4.2	Evaluarea modului de recunoaștere facială	10
4.2.1	Comparația clasificatorilor	10
4.2.2	Hiperacordarea (Hyperparameter Tuning) și Optimizarea SVM	10
5	Concluzii și Direcții Viitoare	11
5.1	Realizări și îndeplinirea obiectivelor	11
5.2	Contribuții proprii	11
5.3	Observații privind rezultatele obținute	11
5.4	Direcții de dezvoltare viitoare	11

1 Introduction

1.1 Contextul temei și probleme rezolvate

Tema proiectului constă în dezvoltarea unui sistem capabil să detecteze fețele umane dintr-un flux video live folosind exclusiv tehnici „pure” de procesare de imagini, bazate pe formule matematice din geometrie și pe transformări în anumite spații de culoare. Implementarea algoritmilor de detecție este realizată în limbajul C++, fiind urmată de o etapă de recunoaștere facială dezvoltată în Python, ce utilizează algoritmul clasic *Eigenfaces* în combinație cu un clasificator de tip Support Vector Machine (SVM). De asemenea, performanța sistemului și rezultatele obținute în urma implementării au fost validate prin teste și metrici de evaluare.

1.2 Obiectivele proiectului

Principalul obiectiv al acestui proiect este crearea unei aplicații capabile să recunoască fețele umane în timp real, sub un set de condiții specifice. Din punct de vedere funcțional, scenariul de utilizare simulează procesul de înregistrare și autentificare biometrică întâlnit pe dispozitivele mobile inteligente (cum este etapa de configurare inițială a recunoașterii faciale). Proiectul își propune să realizeze un echivalent conceptual al tehnologiei FaceID, adaptat însă la constrângerile algoritmilor clasici de procesare numerică a imaginilor (fără a apela la rețele neuronale). Deși sistemul propus prezintă o complexitate și o precizie mai reduse în comparație cu soluțiile comerciale hardware, acesta oferă o alternativă software.

2 Fundamentare Teoretică

2.1 Detecția facială

Pentru rezolvarea problemei de localizare a feței umane într-o imagine, literatura de specialitate oferă o gamă variată de soluții bazate pe inteligență artificială, învățare automată sau procesare digitală de imagini. Printre cele mai cunoscute metode orientate pe extragerea de caracteristici se numără:

- **Algoritmul Viola-Jones** O metodă extrem de eficientă în timp real, bazată pe caracteristici de tip Haar, imagini integrale pentru calcul rapid și un clasificator în cascadă antrenat prin tehnica AdaBoost <https://www.mygreatlearning.com/blog/viola-jones-algorithm/#what-is-viola-jones-algorithm>.
- **Histogram of Oriented Gradients (HOG)** O metodă stabilă care numără aparițiile orientărilor gradientului în regiuni localizate dintr-o imagine, trăsăturile rezultate fiind clasificate ulterior cu ajutorul unei Mașini cu Vector Suport (SVM) <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1467360>.
- **Local Binary Patterns (LBP)** Un operator simplu, dar foarte eficient, care etichetează pixelii unei imagini prin binarizarea vecinătății fiecărui pixel, generând histograme utilizate pentru detecție și recunoaștere <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=5972286>.

În cele din urmă, am folosit un algoritm bazat doar pe procesare de imagini, care transformă imaginea din spațiul de culoare RGB în spațiile de culoare HSV și YCbCr[3]. Spațiul HSV este cel studiat în cadrul laboratoarelor, unde H reprezintă unghiul, S reprezintă saturația, iar V reprezintă intensitatea luminoasă[5]. În spațiul de culoare YCbCr, Y reprezintă luminozitatea, Cb este diferența față de albastru, iar Cr este diferența față de roșu[4]. Formulele de conversie au fost stabilite prin rezolvarea unui sistem liniar de ecuații, rezultând relația matematică directă dintre aceste spații. În urma acestor transformări, care au loc independent una de alta, se aplică o binarizare pe imagine pentru a determina mulțimea pixelilor care respectă condiția de a fi asemănători cu culoarea pielii.

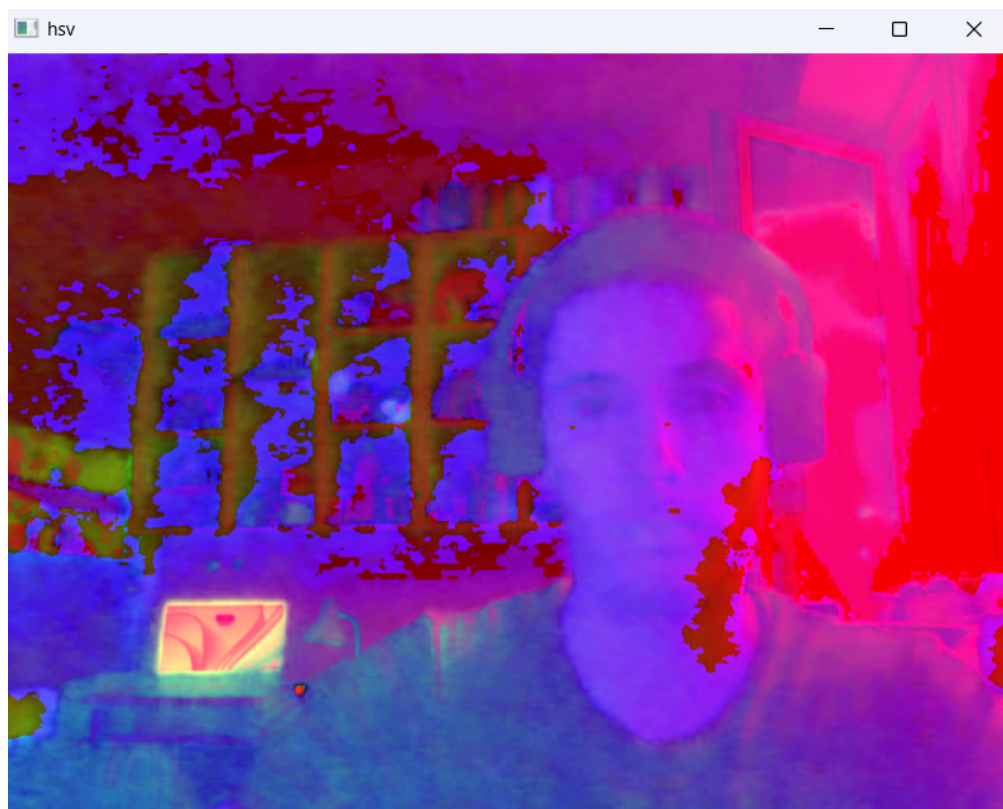


Figure 1: Imaginea în spațiul de culori HSV.

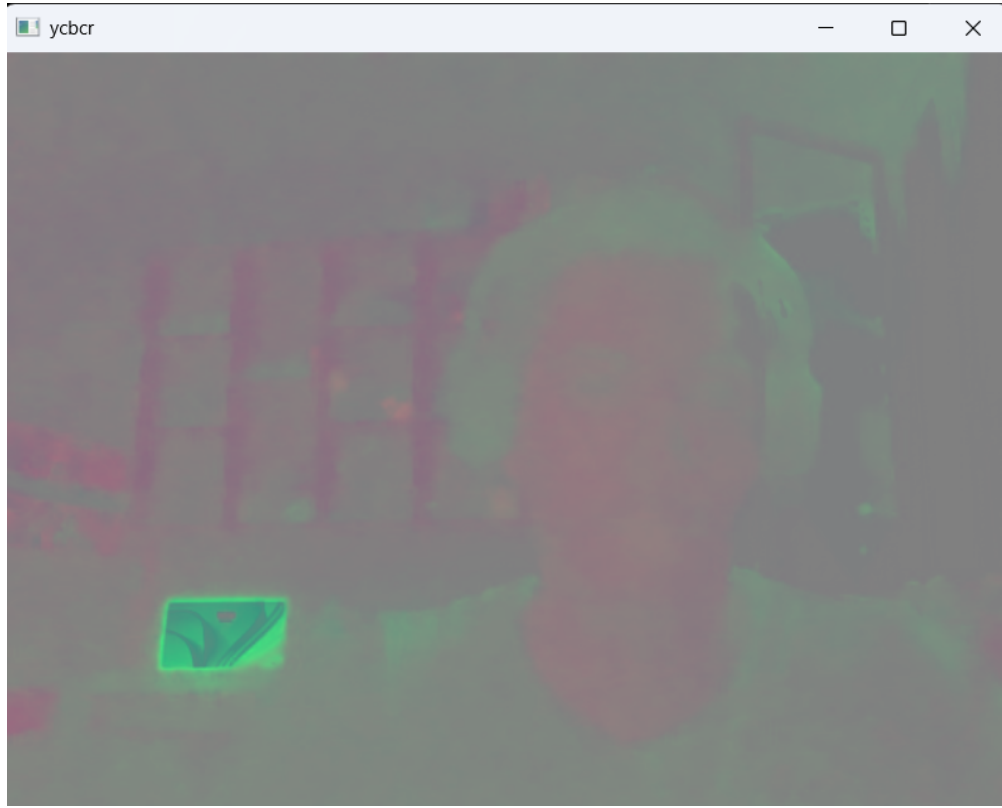


Figure 2: Imaginea în spațiul de culori YCbCr.

2.2 Recunoasterea fetei:

După izolarea regiunii de interes (ROI) și redimensionarea feței decupate la dimensiunea standardizată de 100×100 pixeli, pipeline-ul continuă cu etapa de identificare a persoanei. În cadrul metodelor clasice bazate pe învățare automată, algoritmul *Eigenfaces* reprezintă o soluție fundamentală și extrem de robustă[2, 1].

Algoritmul se bazează pe Analiza Componentelor Principale (PCA) aplicată asupra setului de imagini de antrenament[6]. Imaginile bidimensionale sunt aplatizate într-un vector unidimensional masiv de 10.000 de elemente (100×100). Pentru a extrage trăsăturile matematice esențiale și a reduce dimensionalitatea datelor, algoritmul parcurge următorii pași:

1. Se calculează fața medie a întregului set de date și se scade din fiecare imagine pentru a păstra doar diferențele unice ale fiecărui individ.
2. Se determină matricea de covarianță și se calculează valorile proprii (*eigenvalues*) și vectorii proprii (*eigenvectors*) asociați acestora[6].
3. Vectorii proprii cu cele mai mari valori proprii sunt reținuți (formând spațiul fețelor proprii sau *Eigenfaces*), reducând spațiul inițial la un vector compact de caracteristici principale ($K = 150$).

În final, vectorul redus obținut prin proiecția imaginii în spațiul PCA este transmis clasificatorului supervizat de tip **Support Vector Machine (SVM)**. Spre deosebire de abordările simple bazate pe distanța euclidiană minimă (cum este KNN), clasificatorul SVM optimizează procesul prin calcularea unui hiperplan de separare maximă între clase într-un spațiu multidimensional.

3 Proiectare si implementare

În acest capitol este detaliat pipeline-ul complet de procesare a imaginilor.

3.1 Descrierea solutiei alese si a modului de implementare

Dupa cum remarcam, solutia aleasa este reprezentata de implementarea algoritmului ce transforma in spatiul YCbCr si HSV, urmat de o binarizare si algoritmul clasic, eigenfaces.

Procesul pornește de la fluxul video live preluat de la camera web. Fiecare cadru este analizat la nivel de pixel prin conversia din spațiul nativ RGB în spațiile de culoare alternative HSV și YCbCr. Etapa următoare o reprezintă binarizarea imaginii pe baza proprietăților cromatice ale pielii umane. Pragurile matematice utilizate pentru filtrarea pixelilor de tip piele sunt următoarele:

Spațiu de culoare	Parametri	Condiții
HSV	Hue (H)	$0.0 \leq H \leq 50.0$
	Saturation (S)	$0.23 \leq S \leq 0.68$
RGB	Valori absolute	$R > 95, G > 40, B > 20$
	Relații	$R > G \wedge R > B$
	Diferență	$ R - G > 15$
	Alpha	$A > 15$
YCbCr	Valori absolute	$Y > 80, Cr > 135, Cb > 85$
	Linie 1	$Cr \leq 1.5862 \cdot Cb + 20$
	Linie 2	$Cr \geq 0.3448 \cdot Cb + 76.2069$
	Linie 3	$Cr \geq -4.5652 \cdot Cb + 234.5652$
	Linie 4	$Cr \leq -1.15 \cdot Cb + 301.75$
	Linie 5	$Cr \leq -2.2857 \cdot Cb + 432.85$

Table 1: Condiții pentru detectarea pixelilor de piele.

Condiția globală:

$$\text{pixel} = \text{piele} \iff (HSV) \vee (RGB \wedge YCbCr) \quad (1)$$

În urma aplicării acestei logici, se obține o imagine binară. Pentru a elimina zgomotul generat de camera video, pe această imagine se aplică un **filtru median**. Ulterior, se rulează o operație morfologică de **închidere** (dilatare urmată de eroziune) cu scopul de a umple golurile din interiorul regiunilor compacte de piele și de a elimina micile particule izolate din fundal. Din imaginea rezultată, a fost aplicat algoritmul clasic de BFS, cu vecinătatea de 8 pixeli pentru a determina componentele conexe. Apoi, pentru a încerca să eliminăm false positive-urile, am încercat să filtrăm obiectele, în funcție de relații matematice:

- Area : am ales ca area componentei conexe să fie mai mare de 4500 de pixeli și mai mică de 200000 de pixeli.
- Width on height: raportul dintre lățime și lungime să fie mai mare decât 0.4 și mai mic decât 1.0 .

Fiecare componentă conexă validată de aceste filtre este considerată o față umană. Regiunea corespunzătoare (ROI) este decupată, convertită în nuanțe de gri și salvată pe disc sub numele **test_face.png**.

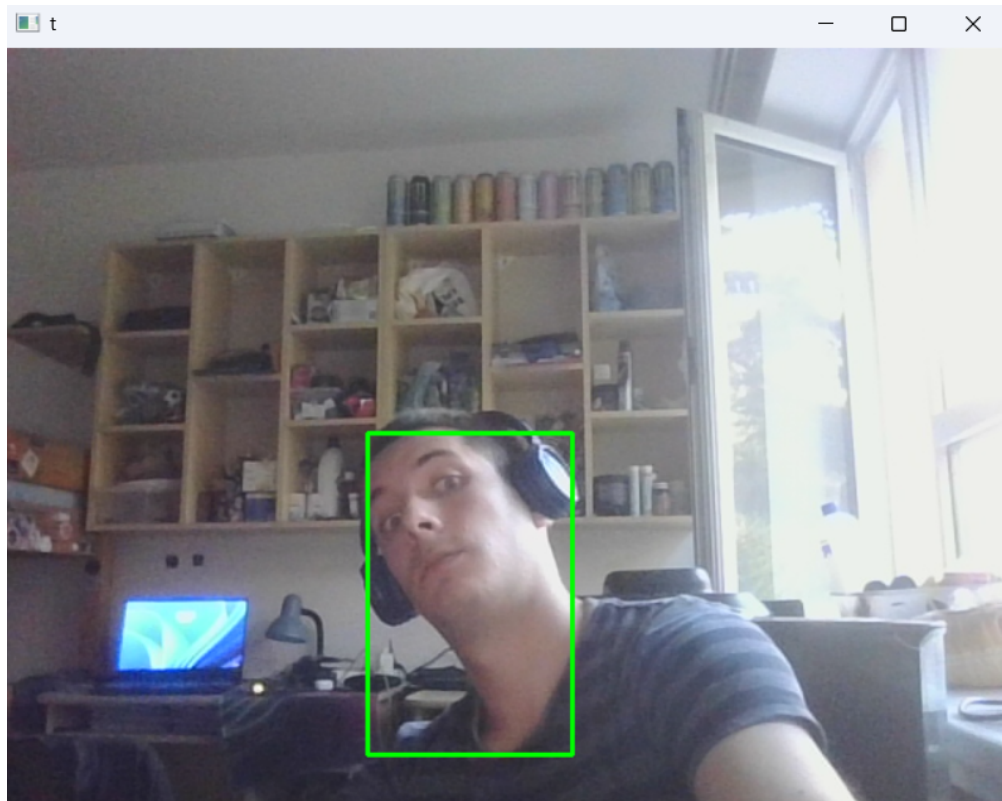


Figure 3: Rezultatul detectării feței.

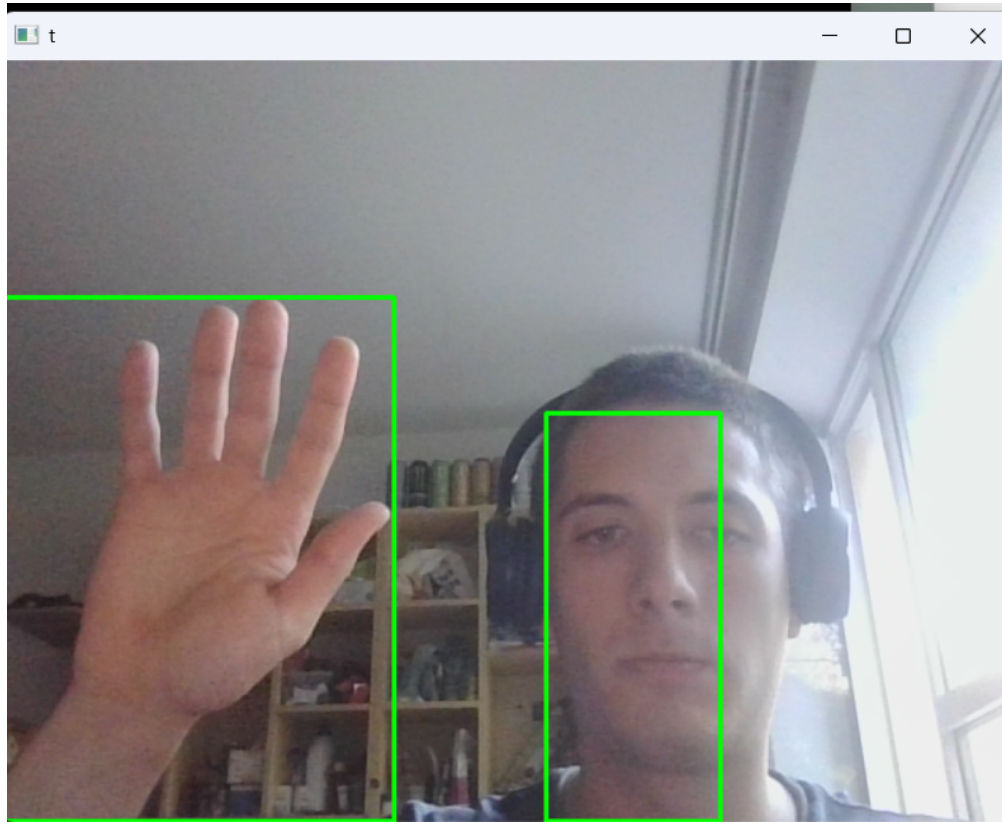


Figure 4: Rezultatul detectării feței cu false positive.

3.2 Descrierea funcționalităților modulelor implementate

Aplicația este structurată în două module independente care comunică prin intermediul sistemului de fișiere:

1. **Modulul de Detecție (C++):** Este responsabil cu accesarea cadrelor de la cameră, realizarea transformărilor matematice de culoare, eliminarea zgomotului, rularea BFS și desenarea dreptunghiurilor verzi de încadrare. Pentru a preveni blocarea ferestrei video live, modulul folosește timere active (`std::chrono`). Acesta salvează imaginea feței o dată la 500 ms în modul de scanare și apelează scriptul de recunoaștere o dată la 5 secunde.
2. **Modulul de Recunoaștere și Clasificare (Python):** Se activează la apelul executabilului C++. Acesta încarcă imaginea `test_face.png`, o redimensionează la 100×100 pixeli, o aplatizează într-un vector și o proiectează în spațiul redus de caracteristici ($K = 150$) obținut prin matricea de covarianță PCA (*Eigenfaces*). Modelul SVM antrenat cu kernel RBF prezice identitatea persoanei și scrie numele rezultat în fișierul `prediction.txt`, text care va fi citit instantaneu și afișat pe ecran de către modulul în C++.

3.3 Pipeline-ul Complet al Sistemului

Sistemul implementat urmează o arhitectură modulară în doi pași: detecția facială în C++ pe fluxul video live, urmată de recunoașterea facială în Python pe imaginea decupată. Procesul pornește de la captarea cadrelor video de la camera web. Fiecare cadru este transformat din spațiul RGB nativ în spații de culoare alternative (HSV și YCbCr), unde caracteristicile cromatice ale pielii umane sunt mai bine deliniate. Se aplică o binarizare bazată pe condițiile din Tabelul 1, rezultând o mască binară care identifică pixelii de piele. Această mască este apoi purificată prin filtrare mediană pentru eliminarea zgomotului, urmată de operații morfologice de închidere (dilatare și eroziune) pentru a umple golurile din interiorul regiunilor și a

elimina particulele izolate din fundal. Algoritmul BFS cu 8-conectivitate parcurge imaginea binară și etichează componentele conexe, calculând simultan aria și bounding box pentru fiecare regiune. Filtrele geometrice (arie cuprinsă între 4500 și 200000 pixeli, raport lățime/înălțime între 0.4 și 1.0) selectează doar regiunile care corespund morfologic unei fețe umane, eliminând regiunile fals pozitive din fundal. Prima componentă validată este decupată, convertită în nuanțe de gri și salvată pe disc sub numele `test_face.png`.

În a doua fază, modulul Python prelucrează imaginea decupată pentru recunoaștere. Imaginea este redimensionată la standardul 100×100 pixeli, aplatizată într-un vector de 10.000 de elemente și normalizată prin scăderea mediei setului de antrenament. Vectorul este apoi proiectat în spațiul redus al componentelor principale (Eigenfaces) obținut din analiza PCA a imaginilor de antrenament, reținând primii $K = 150$ vectori proprii cu cele mai mari valori proprii. Această reducere drastică de dimensionalitate (de la 10.000 la 150 caracteristici) elimină zgomotul și reține doar componentele esențiale ale structurii faciale. Vectorul de caracteristici rezultat este transmis unui clasificator SVM antrenat cu kernel RBF, care calculează distanța până la hiperplanul de separare în spațiul caracteristicilor și returnează eticheta clasei corespunzătoare (identitatea persoanei). Rezultatul predicției este scris în fișierul `prediction.txt`, care este citit instantaneu de modulul C++ și afișat pe ecran. Sincronizarea inter-proces se realizează prin intermediul sistemului de fișiere: C++ salvează imaginea feței orice 500 ms în modul scanare/recunoaștere și apelează scriptul Python orice 5 secunde pentru a limita lag-ul și a permite o predicție mai stabilă în timp real.

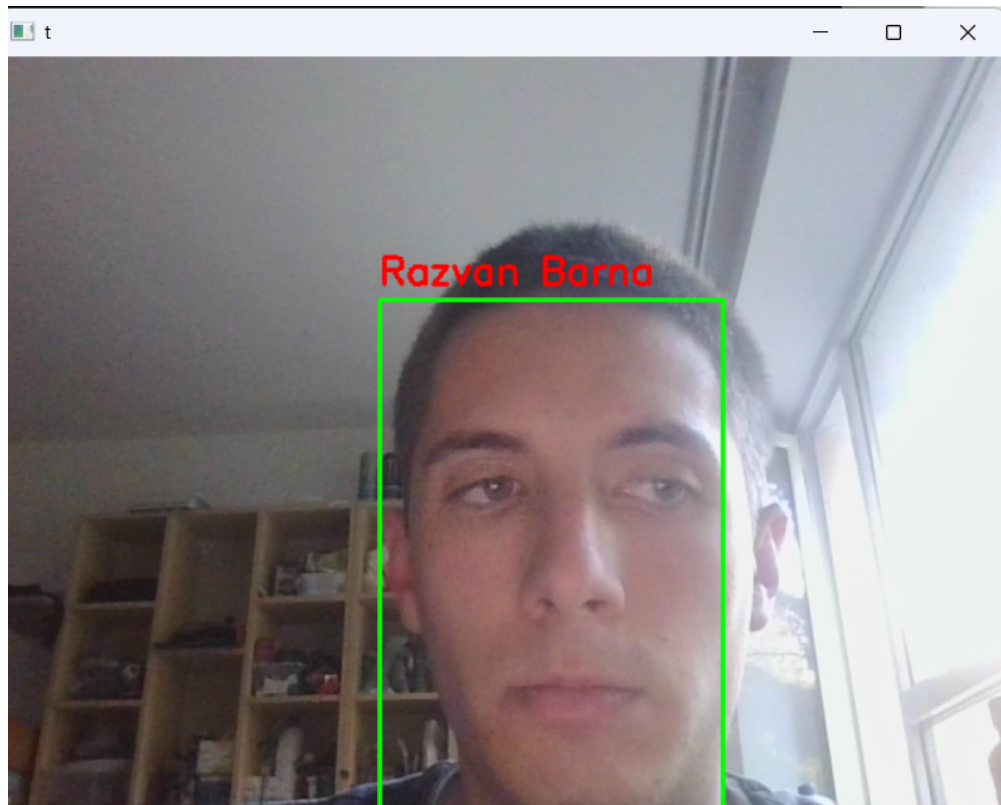


Figure 5: Rezultatul recunoașterii faciale.

3.4 Modul de utilizare al aplicației implementate

Aplicația are 3 funcții principale:

1. Detectia simplă a fețelor captate de camera web: utilizatorul va introduce ca input un număr mai mic sau egal cu 0 când va fi întrebat dacă dorește să i se scaneze fața sau să fie recunoscut.
2. Scanarea feței: utilizatorul va introduce un număr mai mare ca 0, își va introduce numele complet și numărul de poze pe care dorește să le facă programul cu fața sa. În această etapă, va trebui să „pozeze”

sub diferite ipostaze, formându-se o arie variată și ar fi de preferat ca numărul de poze să fie cât mai mare.

3. Recunoașterea feței: Va trebui introdus un număr mai mare ca 0 la întrebarea de predicție, iar programul va face o predicție a tuturor fețelor detectate de cameră, o dată la 5 secunde, pentru a limita lag-ul.

4 Rezultate Experimentale

În acest capitol sunt prezentate și analizate rezultatele obținute în urma testării independente a celor două module principale ale sistemului: modulul de detecție facială bazat pe procesare digitală de imagini (C++) și modulul de recunoaștere facială bazat pe tehnici de învățare automată clasică (Python). Evaluarea performanțelor a fost realizată atât calitativ, prin analiza comportamentului în timp real, cât și cantitativ, utilizând seturi de date de test și metrici.

4.1 Evaluarea modulului de detecție facială

4.1.1 Analiză calitativă în timp real

Din punct de vedere calitativ, algoritmul implementat în C++ (bazat pe segmentare cromatică YCbCr/HSV și filtrare geometrică[3]) oferă rezultate excelente în condiții bine alese. Dacă fundalul scenei nu conține obiecte sau regiuni ale căror nuanțe să se intersecteze cu spectrul culorii pielii (cum ar fi elemente de mobilier din lemn masiv sau pereți în nuanțe calde/bej) și subiecții se află la o distanță acceptabilă față de cameră (aproximativ 0.5 – 2 metri), fețele sunt încadrate stabil și precis de dreptunghiurile de detecție.

4.1.2 Pipeline-ul de testare automatizată

Pentru evaluarea cantitativă riguroasă, s-a utilizat un set de date de test complex, având adnotările originale în format YOLO[7]. Deoarece procesul manual de testare ar fi fost inefficient, a fost dezvoltat un pipeline de automatizare inter-procese utilizând un script Python.

Fluxul de testare a urmat următorii pași:

1. Scriptul Python a convertit adnotările de tip YOLO în coordonate absolute de tip bounding box pentru a servi drept *Ground Truth*.
2. Pentru fiecare imagine din setul de test, scriptul Python a apelat fișierul executabil compilat în C++ (`OpenCVApplication.exe`), transmițând calea imaginii ca argument.
3. Executabilul C++ a procesat imaginea, a rulat algoritmi de detecție și a salvat coordonatele fețelor detectate într-un fișier centralizat în format `.csv`.
4. În final, scriptul Python a încărcat fișierul `.csv` generat și a calculat metricile statistice, comparând detecțiile proprii cu cele obținute prin algoritmul nativ din OpenCV, *Haar Cascade Classifier*.

4.1.3 Rezultate cantitative și comparație (IoU vs. Precizie)

Testarea a fost realizată pe un set de date dificil, caracterizat prin imagini cu fețe multiple, dimensiuni variate, rotații ale capului și fundaluri complexe[7]. Evaluarea s-a bazat pe metrica *IoU* (*Intersection over Union*), stabilind un prag de succes pentru a considera o detecție ca fiind adevărat pozitivă (*TP*).

Rezultatele obținute de algoritmul propriu în funcție de pragul *IoU*, comparativ cu *Haar Cascade*, sunt în Tabelul 2.

Table 2: Performanța detecției faciale în funcție de pragul IoU

Algorithm / Model	Prag IoU	Precizie Obținută
Algoritm Propriu (Procesare Pură)	$IoU \geq 0.5$	0.25 (25%)
Algoritm Propriu (Procesare Pură)	$IoU \geq 0.1$	0.45 (45%)
Haar Cascade Classifier (OpenCV)	Orice prag	0.60 (60%)

Comentariul rezultatelor:

- La un prag strict de $IoU \geq 0.5$, algoritmul propriu obține o precizie de 0.25. Această valoare relativ scăzută este justificată de natura procesării pure de imagini: masca binară obținută prin segmentare cromatică tinde să includă parțial gâtul, urechile sau firele de păr, determinând o decupare a bounding box-ului ușor mai largă sau deformată față de chenarul ideal strâns din adnotările YOLO.
- În momentul în care pragul de validare este relaxat la $IoU \geq 0.1$, precizia sistemului crește semnificativ la 0.45 (45%). Această dinamică demonstrează din punct de vedere matematic faptul că **algoritmul localizează corect poziția spațială a fețelor în imagine** („are habar” unde se află acestea), chiar dacă geometria chenarului nu se suprapune perfect pe masca ideală.
- Prin comparație, modelul *Haar Cascade* menține o precizie constantă de aproximativ 0.60 indiferent de variațiile fine de prag, beneficiind de un set masiv de caracteristici Haar preantrenate, însă cu un cost computațional diferit și fără control direct asupra componentelor geometrice ale pixelilor, așa cum oferă soluția nativă implementată în C++.

4.2 Evaluarea modului de recunoaștere facială

Pentru etapa de recunoaștere facială, evaluarea s-a realizat pe un set separat de date dedicat recunoașterii de identități (*Face Recognition Dataset*)[8]. Imaginile au fost standardizate la dimensiunea de 100×100 pixeli și reduse ca dimensionalitate prin tehnica *Eigenfaces* (PCA)[2, 1, 6], păstrând primele $K = 150$ componente principale.

4.2.1 Comparația clasificatorilor

Au fost testați în mod comparativ mai mulți algoritmi de învățare automată supervizată pentru a identifica modelul optim de clasificare a vectorilor de trăsături PCA:

- *K*-Nearest Neighbors (*KNN*)
- Random Forest (*RF*)
- XGBoost
- Support Vector Machine (*SVM*)

În urma fazei inițiale de testare, s-a constatat că modelele *KNN*, Random Forest și XGBoost au suferit fenomene accentuate de supraantrenare (*overfitting*) sau au oferit rezultate slabe pe setul de validare, din cauza dimensionalității mari și a numărului relativ restrâns de eșantioane per clasă. Singurul clasificator care a demonstrat stabilitate și o capacitate reală de generalizare a fost **Support Vector Machine (SVM)**.

4.2.2 Hiperacordarea (Hyperparameter Tuning) și Optimizarea SVM

Pentru maximizarea performanței modelului SVM, a fost aplicată o procedură de hiperacordare fină (*Grid Search* cu *Cross-Validation*). S-au testat diverse tipuri de funcții nucleu (*kernels*), parametrii de regularizare (C) și coeficienții de scalare (γ).

În urma optimizării, s-au desprins următoarele configurații critice:

- **Kernel utilizat:** Cel mai bun rezultat a fost obținut utilizând un nucleu de tip **RBF** (*Radial Basis Function*). Acesta permite separarea non-liniară a claselor în spațiul caracteristicilor geometrice formate de fețele umane.
- **Acuratețea finală:** După aplicarea scalării datelor (*StandardScaler*) și a parametrilor optimi prin procesul de hipertuning, modelul SVM a atins o acuratețe generală de **aproximativ 60%** pe setul de testare pentru recunoaștere facială.

Având în vedere că recunoașterea se realizează pe un spațiu de caracteristici liniare brute generate prin proiecția PCA a pixelilor, o acuratețe de 60% validează matematic modelul hibrid propus, fiind un rezultat acceptabil.

5 Concluzii și Direcții Viitoare

5.1 Realizări și îndeplinirea obiectivelor

Toate obiectivele propuse inițial au fost îndeplinite prin îmbinarea tehnicilor clasice de procesare a imaginilor cu algoritmi de SVM.

Pipeline-ul dezvoltat demonstrează că este posibilă realizarea unui sistem funcțional de tip „autentificare biometrică” (similar conceptual tehnologiei FaceID) fără utilizarea unor rețele neuronale adânci, adesea costisitoare din punct de vedere computațional.

5.2 Contribuții proprii

Printre principalele contribuții aduse în dezvoltarea acestei aplicații se numără:

- **Proiectarea unui pipeline robust de binarizare:** Integrarea spațiilor de culoare YCbCr și HSV pentru a crea o mască cutanată imună la variațiile moderate de lumină, urmată de optimizarea filtrelor morfologice (eroziune) pentru eliminarea zgomotului și aplicarea filtrului median.
- **Implementarea algoritmului BFS pentru ROI:** Adaptarea parcurgerii în lățime pentru etichetarea componentelor conectate și calculul direct al bounding-box-urilor, urmat de filtrarea geometrică bazată pe factorul de subțirime (*thinness ratio*) și raportul de aspect.

5.3 Observații privind rezultatele obținute

În urma rulării testelor și a evaluării metricilor calculate, s-au desprins următoarele concluzii:

1. **Sensibilitatea la mediu:** Fiind un algoritm bazat pe procesare pură de imagini, calitatea segmentării depinde direct de iluminat și de uniformitatea fundalului. Prezența unor obiecte cu nuanțe similare pielii poate introduce regiuni fals pozitive, însă acestea sunt parțial eliminate prin constrângerile geometrice de arie și aspect din funcția *draw_with_thinness*.
2. **Eficiența compromisului de timp:** Limitarea predicției la un interval de 5 secunde s-a dovedit a fi o soluție acceptabilă, chiar dacă imaginile se mișcă sacadat.
3. **Performanța Eigenfaces + SVM:** Reducerea dimensionalității imaginilor decupate (100×100 pixeli în vectori de caracteristici PCA) reține componentele principale esențiale ale feței.

5.4 Direcții de dezvoltare viitoare

Deși sistemul curent este stabil și îndeplinește criteriile academice impuse, performanțele sale pot fi extinse în viitor prin următoarele îmbunătățiri:

- **Data Augmentation automat:** Includerea în scriptul de Python a unor transformări automate (rotiri ușoare, modificări controlate de contrast) pe cele 30 de imagini salvate inițial, pentru a mări baza de date și a crește robustețea modelului SVM la înclinarea capului.
- **Tranziția către Deep Learning pentru detecția facială:** Înlocuirea algoritmului bazat pe segmentare cromatică (YCbCr/HSV) și morfologie matematică cu modele deep learning ușoare și optimizate pentru rulare în timp real.
- **Utilizarea de Deep Embeddings pentru recunoaștere:** Înlocuirea metodei clasice *Eigenfaces* cu o rețea neuronală convoluțională preantrenată pentru recunoaștere facială. Acest vector de trăsături de înalt nivel poate fi apoi clasificat mult mai precis utilizând modelul SVM deja implementat în scriptul *train_model_svm.py*.

References

- [1] Video foarte bun cu explicatia la eigenFaces, <https://www.youtube.com/watch?v=jQ0ZrXZTXcw>
- [2] Paper EigenFaces, <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=139758>
- [3] Paper Alogritm FaceDetection HSV si YCBCR, <https://www.sciencedirect.com/science/article/pii/S1877050915018918>
- [4] Paper Alogritm YCBCR FaceDetection, <https://www.sciencedirect.com/science/article/pii/S1877050915018918>
- [5] Paper Alogritm HSV FaceDetection, https://www.researchgate.net/publication/262275791_Human_face_detection_system_using_HSV
- [6] EigenFaces Algorithm, <https://www.geeksforgeeks.org/machine-learning/ml-face-recognition-using-eigenfaces-pca-algorithm/>
- [7] Dataset Face Detection, <https://www.kaggle.com/datasets/fareselmenshawii/face-detection-dataset>
- [8] Dataset Face Recognition, <https://www.kaggle.com/datasets/vasukipatel/face-recognition-dataset>